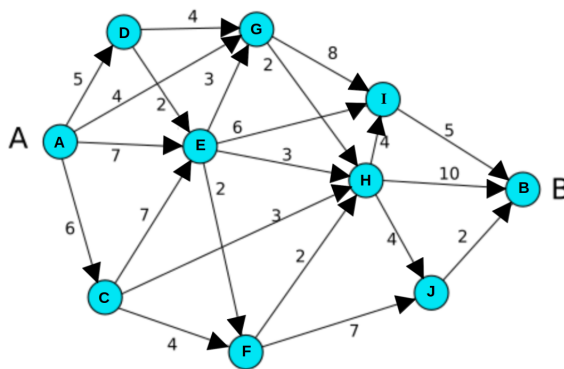


This problem set is due by 11:59pm on **Friday, Feb 7**. It should be submitted through Canvas. All submissions should be typeset using LaTeX.

Problem 1 (Learning paradigms) In the first lecture, we discussed several different learning paradigms. For each example, state whether the learning approach is value-based, policy-based, model-based, or some combination of the three.

- a) **(1 point)** An unmanned ground vehicle (UGV) is tasked with navigating challenging terrain. To accomplish this, you use a simple dynamics model governed by nonholonomic kinematics. Through a supervised learning approach, you learn a mapping from a sequence of images to a traversability metric. This metric then guides the evaluation of your policy at runtime.
- b) **(1 point)** You desire for an unmanned aerial vehicle (UAV) to land at a precise location. You use a model of the system to design an optimal policy which you then execute at run-time. During each execution of the policy, you update the parameters of a neural network to compensate for model-mismatch. You run trials until convergence.
- c) **(1 point)** You design a robot navigation algorithm. The algorithm is intended to learn online in the real-world. The algorithm functions by learning a quality function from state-action pairs observed at runtime. It then uses this quality function to update policy parameters before taking a new action.
- d) **(1 point)** You build a new control algorithm for a robot manipulator. The algorithm uses an initial policy to repeatedly execute a manipulation task. Using a batch of trials, the algorithm then fits the local linear model, which is in turn used to compute an estimate of the cost-to-go and an associated stochastic policy using iterative LQG. The new stochastic policy is then executed on the robot to collect a new batch of data.

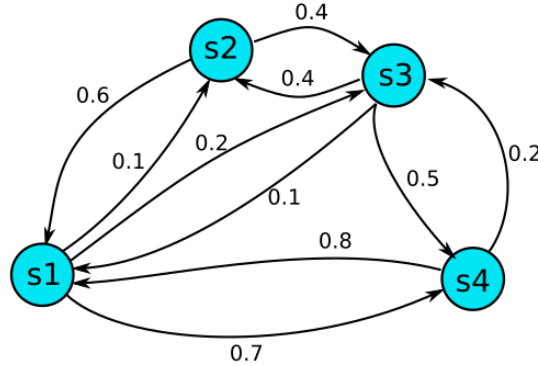
Problem 2 (Shortest Path Problem) Consider the graph below. We assume that the graph is traversed from left to right, with each node only being visited once.



- a) (3 point) Use dynamic programming by hand to compute the optimal policy from A to B. Write down the node sequence and the value of the cost-to-go from A.
- b) (1 points) Solve the problem using a “greedy” approach. Write down the node sequence and the value of the cost-to-go from A.
- c) (2 points) In a paragraph, summarize the difference(s) between the dynamic programming algorithm and Dijkstra’s algorithm (https://en.wikipedia.org/wiki/Dijkstra%27s_algorithm).

Problem 3 (Infinite Horizon Dynamic Programming) In this problem, we explore value iteration and policy iteration.

- a) (1 points) Run the MATLAB file “grid_world_value_iteration_finite_time.m”. For the given grid world problem, how many values are needed to store a time-varying cost-to-go for a 20 time-step horizon. Hint: Try running this algorithm for a few number of iterations (e.g., 5).
- b) (1 points) Run the MATLAB file “grid_world_value_iteration.m”. Assuming the cost-to-go converges, how many values are required to store the cost-to-go. Is the number of values the same or different as in 3a)? Why?
- c) (2 points) Consider the Markov decision process:



The policy evaluation equation for this system is given as

$$\vec{J}^\pi = (\mathbf{I} - \gamma \mathbf{P})^{-1} \mathbf{g}. \quad (1)$$

Write down the transition matrix $\mathbf{P}$ for this system.

- d) (2 points) Modify the MATLAB file “grid_world_value_iteration.m” for a stochastic process model (Markov decision process) where the probability of moving in the direction of the action is 0.6 and moving in a different direction is 0.1. For instance, if you are in state s_0 , $p(s_1|s_0, a_1) = 0.6$ and $p(s_2|s_0, a_1) =$

$p(s_3|s_0, a_1) = p(s_4|s_0, a_1) = p(s_0|s_0, a_1) = 0.1$. Implement 'function $J_0 = \text{expected_cost_to_go}(xis, a, J)$ ' and modify the function 'bellman_backup'. Name your modified file "grid_world_value_iteration_stochastic.m". Submit your code and plots of the cost-to-go and the resulting policy.

- e) **(4 points)** Use the MATLAB file "grid_world_value_iteration.m". Modify this code to implement modified policy iteration with 100 iterations of policy evaluation. Plot the estimated value function as a function for each update step and compare with value iteration. Explain any difference in convergence rate. Name your modified file "grid_world_policy_iteration.m". Also submit your code and plots of the cost-to-go and policy.

Problem 4 (Value Iteration for Continuous State Spaces) In class, we considered the optimal minimum-time solution for the double integrator system generated using value iteration. In this problem, we will explore the case where we have a quadratic cost on state and input.

- a) **(2 points)** Use the MATLAB file "brick_vi_mintime.m". Change the cost function to that of the quadratic regulator problem:

$$g(x, \dot{x}, u) = \frac{1}{2}q_p x^2 + \frac{1}{2}q_d \dot{x}^2 + \frac{1}{2}r u^2.$$

for $(q_p = q_d = 1, r = 10)$. Name your modified code "brick_vi_quadratic.m". Submit your code and plots of the control and cost-to-go functions.

- b) **(2 points)** Compute the exact LQR solution (for the continuous-time system) using the lqr command in matlab. Submit your code and plots of the control and cost-to-go functions. Explain any differences you see between the two solutions.
- c) **(3 points)** Modify the value iteration code to achieve swing-up and stabilization of the simple pendulum given by the equation

$$ml^2\ddot{\theta} + b\dot{\theta} + mgl \sin \theta = u \quad (2)$$

about the upright using a cost function of your choice. Let $m = 1$, $l = 0.5$, $b = 0.1$, $I = 0.25$ and $g = 9.8$. Name your code "pend_vi.m". Submit your code and plots of the control and cost-to-go functions.

Problem 5 (Iterative LQR) In Lecture 5, we discussed iterative LQR. In this problem we will implement iterative LQR in MATLAB for the cart-pole system using the MATLAB file “`iterative_lqr_cart_pole.m`”. We will start by implementing the “`backward_pass`” function.

- a) (2 points) For the quadratic cost-function specified, derive g_x , g_u , g_{xx} , g_{ux} , and g_{uu} . Implement this in the code. Note, the gradients for the cost-function are separated into a final cost and running cost-function.
- b) (5 points) Derive the “ Q -terms”. Find $Q_{x,k+1}$, $Q_{u,k+1}$, $Q_{xx,k+1}$, $Q_{ux,k+1}$, $Q_{uu,k+1}$ that comprise the quadratic approximation of the Q -factor for the next timestep (see the right side of equation 5.16 in the notes). Remember $Q_k(x, u) = g_k(x, u) + V_k(x)$. Show your work. Implement in the code in the “ Q -terms” function.
- c) (2 points) Define the gains K and v using the “ Q -terms”. Implement this in the “`gains`” function.
- d) (5 points) Derive the “ V -terms”. Group like terms on the right and left-hand side of equation 5.16 to solve for the quadratic approximation of V_k for the current timestep defined by $V_{x,k}$ and $V_{xx,k}$ in terms of the “ Q -terms” and the gains. Also specify the initial conditions for $V_{x,N}$ and $V_{xx,N}$ for the backward pass. Show your work and implement the “ V -terms” function.
- e) (1 points) Implement the control action u for the forward pass in the “`forward_pass`” function.
- f) (5 points) Choose values for the cost-function Q , R , and Q_f to get a successful swing-up policy for the cart-pole system. Submit your code and plots of the resulting trajectory.